Scalable E-Commerce Big Data Mining and Analytics Platform

Detailed Project Report — Phases 1–7

ASBD — Analysis of Structured & Big Data
Dataset: E-Commerce Behavior Data from Multi-Category Store (October 2019)

Implementation stack: Hadoop HDFS · Apache Spark / PySpark · Parquet · Python · Matplotlib / Seaborn · Plotly

Prepared as a reproducible technical report and project showcase companion. Reported figures are based on the project execution notes and validated outputs available at preparation time.

Executive summary

This project builds a scalable analytics workflow over a large e-commerce event log using HDFS for distributed storage and PySpark for distributed transformation and aggregation. The work completed through Phase 7 covers dataset selection and ingestion, cleaning and data-quality assessment, outlier analysis, feature transformation, feature reduction/encoding assessment, and descriptive analytics. The showcase is intended to be reproducible: heavy data processing remains in Spark, while the dashboard consumes compact CSV summaries and saved visualizations.

Metric	Recorded result
Raw event records	42,448,764
Cleaned records	42,349,874
Processed schema	17 columns after cleaning; 16-column reduced Phase 6 dataset
Phase 5 transformed dataset	21 columns
Cleaned Parquet output	~2.1 GB; 67 Snappy Parquet files
Event types	view, cart, purchase
Distinct users in Phase 7	3,021,435
Distinct sessions (recorded analysis)	9,239,403
Purchase events	742,773
Purchase revenue (sum of purchase-event price/revenue)	2209933,212.63

Project scope and architecture

The source dataset is the October 2019 e-commerce behavior event stream. Each row represents an event (view, cart, or purchase) with product, category, brand, price, user, session, and timestamp fields. It is an event log, not a joined multi-source warehouse. Therefore, the project performs cleaning, feature engineering, reduction, distributed summaries, and behavioral analysis; it does not claim schema integration across independent source systems.

The implemented architecture is: CSV in HDFS → Spark read and validation → cleaned Parquet → transformed and reduced Parquet datasets → distributed aggregations → compact CSV summaries and image/HTML plots → dashboard and report. The local MacBook Air M2 has 8 GB RAM, so Spark is configured for local execution and driver-side collection is restricted to small aggregates or samples.

Phase 1 — Project setup, dataset selection and ingestion

Objective and implementation

Establish a reproducible local big-data environment and make the large source CSV available to HDFS for distributed processing. The project uses Hadoop HDFS for storage and PySpark for processing. The environment includes Java 17, Hadoop 3.5.0 and Spark 4.2.0; the Spark launch command uses local[2] and a 2 GB driver memory limit to remain conservative on the available hardware.

Inputs, outputs and validation

Input: e-commerce behavior CSV for October 2019. HDFS raw path: /ecommerce/raw/2019-Oct.csv. The original dataset was approximately 5.3 GB and contained 42,448,764 records. The project directory separates raw data, processed data, notebooks, source code, results, reports and screenshots.

The ingestion milestone is considered complete in the project tracker. HDFS path checks, Spark readability, schema inspection and row-count checks are the key reproducibility checks. The report does not assert a raw file checksum because none was recorded in the available execution notes.

Phase 2 — Data understanding, selection and initial preprocessing

The dataset was profiled and reduced to attributes relevant to event analytics: timestamp, event type, product and category identifiers, category code, brand, price, user identifier and session. Derived fields include event date/hour, weekday, year/month, purchase indicator, revenue and main category. The cleaned dataset has 17 columns and was written as Snappy Parquet, approximately 2.1 GB over 67 files.

Attribute group	Purpose
Time fields	Temporal trends, hour-of-day and weekday analysis
Event type	Compare views, carts and purchases
Product/category/brand	Product and merchandising analysis
Price/revenue	Value distribution and purchase revenue
User/session	Behavioral aggregation; use with privacy-aware reporting
Derived calendar fields	Efficient grouped analysis and dashboard filters

Phase 3 — Data quality and cleaning

Cleaning applied exact-row de-duplication through dropDuplicates(), filtered out non-positive prices, and filled missing categorical/session fields with the constant Unknown. The cleaned row count is 42,349,874, down by 98,890 rows from the raw count; this difference should not be interpreted as the number of duplicates alone because filtering invalid prices also removes records.

Quality issue	Recorded finding / treatment
Missing category_code	13,515,609 missing before cleaning; filled with Unknown
Missing brand	6,113,008 missing before cleaning; filled with Unknown
Missing user_session	2 missing; filled with Unknown
Invalid price	68,673 records with price <= 0; filtered out
Duplicates	dropDuplicates() used; full independent audit failed due to JVM GC allocation issue
Sample duplicate audit	1% sample (424,129 rows) showed zero duplicates
Post-cleaning nulls	0 across cleaned columns
Post-cleaning invalid prices	0

Important limitation: a complete duplicate audit using exceptAll against dropDuplicates failed with a JVM garbage-collection/allocation error after roughly 166 seconds. A 1% sample had no duplicate rows, but this is not proof that the full dataset contains zero duplicates. The pipeline did execute dropDuplicates(); the report distinguishes the cleaning action from the incomplete independent audit.

Phase 4 — Outlier analysis

Price outliers were assessed using IQR and Z-score methods. IQR quartiles were Q1=66.64 and Q3=358.57, giving IQR=291.93 and an upper fence of 796.465. The method flagged 3,677,875 events (8.69%) above the upper fence and none below the lower fence. The Z-score method used mean 290.7840 and standard deviation 358.3968; |z|>3 corresponds to prices above approximately 1,365.9744, flagging 1,158,750 events (2.74%).

Method	Threshold / statistic	Flagged events
IQR	Upper fence 796.465	3,677,875 (8.69%)
Z-score	Mean 290.7840; SD 358.3968; upper |z| threshold 1,365.9744	1,158,750 (2.74%)

Decision: retain the values in the analytical dataset because a high-priced product may be legitimate rather than erroneous. Outlier flags are diagnostic signals, not automatic deletion rules. IQR flagged events by event type: view 3,510,543; cart 94,171; purchase 73,161.

Phase 5 — Normalization and transformation

The Phase 5 pipeline added min-max normalization, Z-score standardization, decimal scaling and log1p price transformation. The source had 17 columns and the transformed dataset had 21 columns. Baseline price statistics: min 0.77, max 2,574.07, mean 290.7840, standard deviation 358.3968 and skewness 2.5399.

Feature	Formula / interpretation	Observed range or result
price_minmax	(price – min)/(max – min)	0 to 1
price_zscore	(price – mean)/SD	–0.8092 to 6.3708
price_decimal	price / 10,000	0.000077 to 0.257407
price_log	log1p(price)	0.57098 to 7.85363
Skewness	Original vs log transformed	2.53991 vs –0.26836

Validation recorded 42,349,874 rows with no null, NaN or infinite values in the transformed features. The Phase 5 output was written by event-date batches under /ecommerce/processed/ecommerce_oct_2019_phase5. Reading it back emitted a duplicate event_date column warning due to the event_date field also being present as a partition column. The count remained correct; this should be cleaned up or explicitly handled in a future reproducibility pass.

Phase 6 — Feature engineering, encoding assessment and reduction

Categorical cardinalities were event_type=3, main_category=14, category_code=127 and brand=3,446. A Spark ML StringIndexer pipeline was executed with handleInvalid='keep' for these fields as an encoding experiment. The saved reduced Phase 6 dataset retains the original categorical strings; encoded indexes were not persisted as the final analytical representation.

The reduced dataset has 16 columns: event_time, event_type, product_id, category_id, main_category, brand, price, price_log, user_id, user_session, event_date, event_hour, day_of_week, event_month, is_purchase and revenue. It contains 42,349,874 rows and was validated with zero nulls in all 16 fields, zero non-positive prices, consistent event_type/is_purchase mapping and consistent revenue derivation.

Correlation screening identified expected high correlations among price and its derived versions: price with price_minmax, price_zscore and price_decimal at 1.0; price with price_log at approximately 0.8024. event_year was constant at 2019 and excluded from correlation analysis. The reduction decision removed redundant scaled features and retained price_log as a useful transformed attribute.

Phase 7 — Descriptive analytics and visual exploration

Phase 7 generated distributed summary tables, user-behavior aggregates and 14 original plots. Eleven CSV summaries were saved locally in results/phase7, and user-level aggregates were stored in HDFS at hdfs:///ecommerce/results/phase7_user_behavior. The user-behavior table contains 3,021,435 users.

Analysis	Recorded result
Event distribution	View 40,708,807 (96.12%); cart 898,294 (2.12%); purchase 742,773 (1.75%)
Hourly activity peak	21:00; 3,055,061 events
Hourly revenue peak	14:00; 17,531,268.91
Category revenue leaders	Electronics 176,445,577.86; Unknown 22,922,165.08; appliances 13,581,760.58; computers 11,377,870.82
Brand revenue leaders	Apple 111,199,495.70; Samsung 46,402,037.35; Xiaomi 9,192,640.24; Unknown 8,539,747.08
Purchase events and revenue	742,773 events; 229,933,212.63 total purchase revenue
Purchase price	Mean 309.5605; median 179.84; min 0.77; max 2,574.07
Users / sessions	3,021,435 users; 9,239,403 sessions
User activity distribution	1–5 events: 55.23%; 6–10: 14.82%; 11–20: 12.81%; 21–50: 11.25%; >50: 5.88%
Top revenue product	product_id 1005115; revenue 12,404,835.95

The event-level ratios view→cart=2.21%, view→purchase=1.82%, and cart→purchase=82.69% are ratios of event counts, not cohort or session conversion rates. They must not be presented as true conversion probabilities. A correct funnel requires user/session-level stage definitions and ordering.

The project generated 14 Phase 7 PNG plots, 11 summary CSVs, a Phase 7 report, a master tracker, and an artifact ZIP. Additional visualization batches subsequently generated eight PNGs (histogram, scatter, Pareto, area, violin, box, correlation heatmap and radar) and seven more files (doughnut, pair plot, bubble chart HTML, treemap HTML, sunburst HTML, strip plot and funnel HTML). The count of generated plot files is 29 across the original and additional directories; the number of distinct chart types is lower because several types repeat.

Key analytical findings

1. Event traffic is overwhelmingly views, so event counts are not equivalent to unique products or customers. 2. Revenue is concentrated in electronics and a small number of brands, especially Apple and Samsung. 3. Price is strongly right-skewed; log transformation materially reduces the skewness. 4. IQR identifies more price outliers than the stricter |z|>3 method. 5. Most observed users have only 1–5 events, while a smaller group accounts for repeated activity. 6. Time-of-day peaks differ between event volume and revenue, which is why both metrics should be visualized.

Limitations and responsible interpretation

The source covers one month of event-log data, with a UTC timestamp boundary extending into 2019-11-01 in the derived event_date values. Unknown categories are substantial (about 31.82% of category_code and main_category, and 14.27% of brand), so category and brand findings should include that caveat. Event-level funnel ratios are not conversion rates. Some plots use samples or top-N subsets for memory and readability; their titles or accompanying notes should state this. Correlation does not establish causation, and correlations involving derived variables can be structurally high.

The project has completed descriptive and preprocessing stages through Phase 7. It should not claim predictive modelling, association-rule mining, or production deployment as completed in this report. The dashboard should be treated as a presentation layer over precomputed summaries, not as an interface that reprocesses the full 42-million-row dataset on every interaction.

Showcase implementation recommendation

For presentation, use normal modular Python project files for the dashboard and reusable code, with a small optional notebook for educational walkthroughs. A Streamlit dashboard is suitable because it can display KPIs, filterable summaries, PNG plots and Plotly HTML/interactive visualizations with minimal setup. Keep Spark jobs in src/ and keep the dashboard reading precomputed CSV summaries so a reviewer can launch it without starting a full Spark job.

Suggested dashboard sections

Overview: records, users, purchase events, purchase revenue and event distribution. Time trends: hourly activity/revenue and daily purchase revenue. Merchandising: category, brand and product summaries. Customer behavior: user activity groups and aggregated user metrics. Price analysis: histogram, box/violin plots and price buckets. Visualization gallery: additional static PNGs and links to interactive HTML charts. Methodology/limitations: cleaning rules, outlier decision, sampling notes and event-funnel caveat.

Phase status and next steps

Phase	Status	Evidence / next action
1 — Setup and ingestion	Completed	HDFS raw dataset and Spark access
2 — Selection and preprocessing	Completed	Cleaned Parquet, 17-column schema
3 — Data quality	Completed with audit limitation	Full independent duplicate audit failed; sampled audit recorded
4 — Outliers	Completed	IQR and Z-score analyses; retained plausible values
5 — Normalization	Completed	21-column transformed dataset validated
6 — Feature engineering/reduction	Completed	16-column reduced dataset validated
7 — Descriptive analytics	Completed	Summaries, user aggregates, 14 original plots and report
8 — User-feature review	Previously recorded; revalidate	HDFS user features and enriched table exist per project notes
9 — Predictive analytics	Pending	Define target, split strategy, baseline and metrics
10 — Final integration	Pending	Dashboard, final report, reproducibility and demo checklist

Reproducibility checklist

• Confirm Hadoop and Spark versions and HDFS paths before running jobs.

• Use the existing processed Phase 6 Parquet dataset for descriptive summaries.

• Keep full-data aggregations in Spark; collect only compact summaries or bounded samples.

• Store CSV summaries and plots under the project results directory.

• Record execution dates, row counts, schemas and any failed validation explicitly.

• Use the dashboard only with generated summary CSVs and plot artifacts; avoid triggering full-data Spark scans on dashboard startup.

• Keep interactive Plotly charts as HTML; static image export may require Kaleido.

• Do not label event-count ratios as session conversion rates.

End of report. This report documents the state through Phase 7 using the project execution notes and recorded outputs available at preparation time. Before final submission, verify that local artifact folders contain the expected CSVs/plots and regenerate any report sections whose underlying summaries have changed.